

Lab 8 - DLX Print

Nate Herbst
A02307138
Nathan Walker
A02364124

Introduction

In a pipelined processor, there is no built-in communication with it. With just the processor it is a black box that does the computations with no input or output. However, we can add an “exit” to the pipeline that allows us to print to a serial monitor. This includes using the UART from Lab 1 and fifo’s following the execute stage. These fifo’s send the necessary data to the UART while not slowing down the pipeline at all.

Procedure

Modular Systems

The main components that we added were a stack, and division modules. Once these were created it was about integrating them with existing modules

New Module: Stack(stack.vhd)

The Stacks purpose is to reverse the order of integer characters. This is accomplished by putting characters onto the stack by least significant digit to most significant digit. This causes the characters to be in reverse order when the stack is popped.

New Module: Division(division.vhd)

This is an IP block from Quartus. The module was made to have a 8 clock pipeline to make it be able to handle the boards clock frequency.

Top-Level Processor (DLX_Processor.vhd)

Things were changed in the processor to account for detecting the new OPCODES. This included the decode stage to parse the instructions to grab the register for the print statement. Then execute as well to know when to write to the fifo holding the data. Some changes were

made to the writeback stage as well to make sure the processor knew not to write back to a register if it was any of the print statements.

Challenges and Solutions

- **Stack Timing:** Initially the stack and the state machine were mismatched. This caused the stack head pointer to be incorrect. This caused a whole host of issues. The solution was to add some waiting states in our state machine to give fifo and stack time to settle.
- **Repeated Prints:** Once the stack timing was fixed, the issue ran into was our test file repeatedly printed out test character 'Y'. This was an interesting one to debug. We realized that on the physical board, the processor was able to run so fast that it was back tracking in the ROM. This was caused by no done loop at the end of the program. Adding this resolved the issue and our test file only printed the character once.
- **Processor Computation:** Eventually the print commands were able to work properly in the processor able to print the strings and integers correctly. However, once we ran the factorial program, we could not get the correct output. Testing the PD and PDU functions, we verified that it was indeed working properly. We concluded that we did not expect as many downfalls as we experienced therefore running out of time and not being able to debug properly.

Results

The DLX processor with print statements was almost working in hardware. After much trial and error and getting close to it. Further signal tap time would be able to help us determine the root cause of the failure to compute the factorial. Overall the print opcodes work as expected but print the incorrect values due to a part of the processor being broken.

Figures

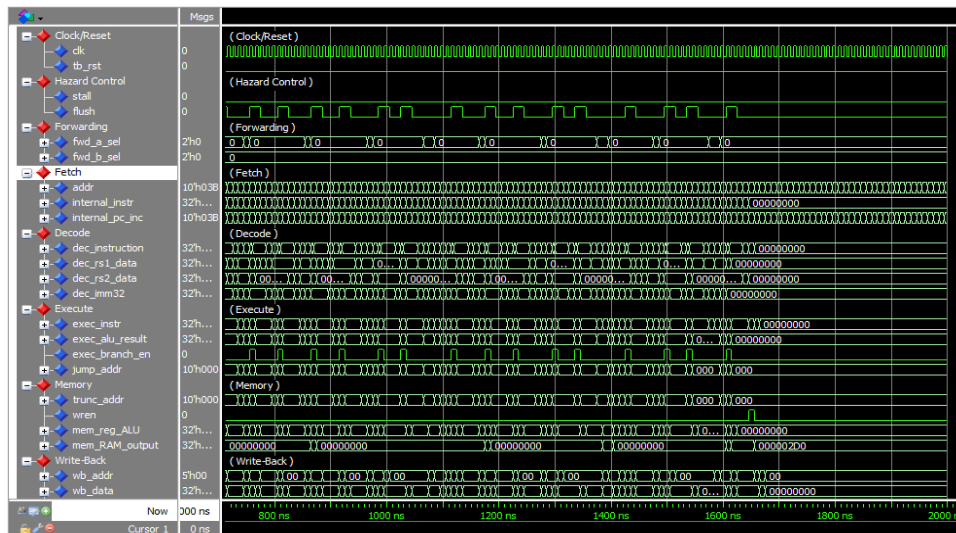


Figure 1: Previous lab simulation showing memory is 2D0 at the end

Conclusion

In this lab, we successfully extended our custom DLX processor and assembler to support three new print instructions—PCH (0x31), PD (0x32), and PDU (0x33)—along with .const segment handling for string constants. The assembler correctly generates MIF files with the new opcodes and embedded string data.

On the hardware side, we implemented and verified the full UART data path: the execute stage triggers FIFO writes for print instructions, the char_translator FSM converts characters and integers to ASCII via a stack and LPM_DIVIDE IP core, and the TX_UART transmits at 19200 baud through the DE10-Lite's Arduino header. We confirmed through bypass testing and incremental debugging that the PLL, dual-clock FIFO, and UART transmitter all function correctly. PCH successfully prints individual characters, PDU correctly outputs unsigned decimal values, and our string printing loop outputs "Welcome to the DLX factorial program!" as expected.

However, we identified that the factorial computation, which worked correctly in Lab 7, now produces an incorrect result (832700416 instead of 24 for 4!). Since the printing logic itself has been verified independently, the issue lies in a pipeline hazard introduced during the modifications to the decode, hazard detection, and forwarding modules needed to support the new print instructions. We have isolated the problem to the multiply subroutine's interaction with the pipeline and plan to resolve it by testing with smaller inputs and using SignalTap to trace register values through the computation.

Appendix

Serial Output when running through our DLX processor

```
Welcome to the DLX factorial program!  
4! = 832700416
```

Lab8_DLX_Print.vhd (top level file)

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
  
library work;  
  
entity Lab8_DLX_Print is  
  port (  
  
    -- Clock  
    ADC_CLK_10 : in std_logic;  
    MAX10_CLK1_50 : in std_logic;  
    MAX10_CLK2_50 : in std_logic;  
  
    -- Tx      :   out std_logic;  
  
    -- Arduino Header  
    -- I/O 0 - Rx  
    -- I/O 1 - Tx  
  
    ARDUINO_IO      : inout std_logic_vector(15 downto 0);  
    ARDUINO_RESET_N : inout std_logic;  
  
    -- Push buttons (active low)  
    KEY : in std_logic_vector(1 downto 0)  
  
  );  
end Lab8_DLX_Print;  
  
architecture component_list of Lab8_DLX_Print is
```

```

component FIFO
  PORT
  (
    data      : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
    rdclk     : IN STD_LOGIC ;
    rdreq     : IN STD_LOGIC ;
    wrclk     : IN STD_LOGIC ;
    wrreq     : IN STD_LOGIC ;
    q         : OUT STD_LOGIC_VECTOR (7 DOWNTO 0);
    rdempty   : OUT STD_LOGIC
  );
end COMPONENT;

```

```

-- Component Declarations

```

```

component PLL_UART
  PORT
  (
    inclk0    : IN STD_LOGIC := '0';
    c0        : OUT STD_LOGIC ;
    c1        : OUT STD_LOGIC
  );
end component;

```

```

component TX_UART
  port
  (
    Pclk : in std_logic;
    Tx: out std_logic;
    rdreq: out std_logic;
    empty: in std_logic;
    data_in: in std_logic_vector(7 downto 0)
  );
end component;

```

```

-- Signals

```

```

--signal Rx : std_logic;
signal Tx : std_logic;

```

```

-- Clock Signals

```

```

signal clk_rx_8x : std_logic; -- 153.6 kHz (8 * 19200)
signal clk_tx_1x : std_logic; -- 19.2 kHz

-- Interconnect Signals
--signal rx_data_byte : std_logic_vector(7 downto 0);
--signal rx_data_valid : std_logic;

constant DATA_WIDTH : integer := 32;

signal tx_data_byte : std_logic_vector(7 downto 0);
signal tx_read_req : std_logic;
signal fifo_empty : std_logic;
signal fifo_full : std_logic;
signal fifo_wr : std_logic;
signal fifo_data : std_logic_vector(DATA_WIDTH-1 downto 0);
signal fifo_instr : std_logic_vector(DATA_WIDTH-1 downto 0);

signal char_wr : std_logic;
signal char : std_logic_vector(7 downto 0);

begin
    ARDUINO_RESET_N <= 'Z';

    -- UART IO Assignments
    --Rx <= ARDUINO_IO(0); -- Read from IO0
    ARDUINO_IO(0) <= 'Z'; -- Tri-state IO0 so it can be used as
input

    ARDUINO_IO(1) <= Tx; -- Write internal Tx to IO1

    ARDUINO_IO(15 downto 2) <= (others => 'Z'); -- Set unused pins to
high-Z

    -- PLL Instantiation
    pll_inst : PLL_UART
    PORT MAP (
        inclk0 => MAX10_CLK1_50,
        c0 => clk_rx_8x,
        c1 => clk_tx_1x
    );

```

```

);

-- TX UART Instantiation
tx_inst : TX_UART
PORT MAP (
    Pclk    => clk_tx_1x,
    Tx      => Tx,
    rdreq   => tx_read_req,
    empty   => fifo_empty,
    data_in => tx_data_byte
);

character_fifo : FIFO
PORT MAP
(
    data      => char,
    rdclk     => clk_tx_1x,
    rdreq     => tx_read_req,
    wrclk     => MAX10_CLK1_50,
    wrreq     => char_wr,
    q         => tx_data_byte,
    rdempty   => fifo_empty
);

char_translator_inst : entity work.char_translator
port map (
    clk => MAX10_CLK1_50,
    fifo_wr => fifo_wr,
    fifo_data => fifo_data,
    fifo_instr => fifo_instr,

    char => char,
    char_wr => char_wr,
    fifo_full => fifo_full
);

--DLX processor
processor_inst : entity work.DLX_Processor
generic map (
    WIDTH => 10,

```

```

        INSTR_WIDTH => 32
    )
    port map (
        clk => MAX10_CLK1_50,
        fifo_full => fifo_full,
        rst => not KEY(0),
        fifo_wr => fifo_wr,
        fifo_data => fifo_data,
        fifo_instr => fifo_instr
    );

end component_list;

```

char_translator.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;

use work.decode_reg_pkg.all;

entity char_translator is
    port (
        clk : in std_logic;
        fifo_wr : in std_logic;
        fifo_data : in std_logic_vector(31 downto 0);
        fifo_instr : in std_logic_vector(31 downto 0);

        char : out std_logic_vector(7 downto 0);
        char_wr : out std_logic;
        fifo_full : out std_logic
    );
end char_translator;

architecture behavioral of char_translator is

```



```

    type state_type is (idle, fifo_ready, compute_div, wait_for_div,
push_wait, wait_once_for_push, wait_for_stack, wait_for_pop,
wait_once_for_idle);
    signal state      :    state_type    := idle;

    signal stack_char  :    std_logic_vector(7 downto 0);
    signal stack_full  :    std_logic;
    signal stack_empty :    std_logic;
    signal char_out    :    std_logic_vector(7 downto 0);
    signal sign_tracker:    std_logic := '0';

    signal rdreq :    std_logic := '0';

    signal data :    std_logic_vector(31 downto 0);
    signal data_rdempty :    std_logic;
    signal data_full :    std_logic;

    signal instr :    std_logic_vector(31 downto 0);
    signal instr_rdempty :    std_logic;
    signal instr_full :    std_logic;

    signal numer :    std_logic_vector(31 downto 0) := (others => '0');
    signal quotient :    std_logic_vector(31 downto 0);
    signal remain :    std_logic_vector(31 downto 0);
    signal temp :    std_logic_vector(31 downto 0) := (others => '0');

    signal div_counter :    integer := 0;
    signal push_counter :    integer := 0;
    signal pop_counter :    integer := 0;

    signal push :    std_logic := '0';
    signal pop :    std_logic := '0';

begin

    fifo_full <= instr_full or data_full;

    data_fifo : entity work.UART_TX_DATA
    port map (
        data => fifo_data,

```

```

        clock => clk,
        rdreq => rdreq,
        wrreq => fifo_wr,
        q => data,
        empty => data_rdempty,
        full => data_full
    );

    instr_fifo : entity work.UART_TX_DATA
    port map (
        data => fifo_instr,
        clock => clk,
        rdreq => rdreq,
        wrreq => fifo_wr,
        q => instr,
        empty => instr_rdempty,
        full => instr_full
    );

    char <= char_out;

    process (clk) begin
        if rising_edge(clk) then
            case state is

                when idle =>
                    rdreq <= '0';
                    push <= '0';
                    pop <= '0';
                    if data_rdempty = '0' and instr_rdempty = '0' then
                        -- Wait 1 cycle for FIFO output register to present
valid data
                        state <= fifo_ready;
                    end if;

                when fifo_ready =>
                    -- FIFO q outputs are now valid (output register has
settled)
                    if instr(31 downto 26) = OP_PD then
                        if data(31) = '1' then

```

```

        temp <= std_logic_vector(-signed(data));
    else
        temp <= data;
    end if;
else
    temp <= data;
end if;
state <= compute_div;

when compute_div =>
    push <= '0';
    if instr(31 downto 26) = OP_PCH then
        state <= push_wait;
        stack_char <= temp(7 downto 0);
        push <= '1';
    else
        numer <= temp;
        state <= wait_for_div;
        stack_char <= "00110000"; --48
    end if;

when wait_once_for_push =>
    push <= '0';
    if push_counter < 2 then
        push_counter <= push_counter + 1;
    else
        state <= wait_for_stack;
        push_counter <= 0;
    end if;

when wait_for_div =>
    push <= '0';
    if div_counter < 8 then
        state <= wait_for_div;
        div_counter <= div_counter + 1;
    else
        if quotient = (quotient'range => '0') and remain =
(remain'range => '0') then
            state <= wait_once_for_push;
        else

```

```

        stack_char(3 downto 0) <= remain;
        push <= '1';
        temp <= quotient;
        state <= push_wait;
    end if;
    div_counter <= 0;
end if;

when push_wait =>
    -- Wait 1 cycle for the stack to process the push
    push <= '0';
    if instr(31 downto 26) = OP_PCH then
        state <= wait_for_stack;
    else
        state <= compute_div;
    end if;

when wait_for_stack =>
    push <= '0';
    pop <= '0';
    if stack_empty = '0' then
        pop <= '1';
        char_wr <= '1';
        state <= wait_for_pop;
    else
        pop <= '0';
        char_wr <= '0';
        rdreq <= '1';
        state <= wait_once_for_idle;
    end if;

when wait_for_pop =>
    pop <= '0';
    char_wr <= '0';
    if pop_counter < 2 then
        pop_counter <= pop_counter + 1;
    else
        state <= wait_for_stack;
        pop_counter <= 0;
    end if;

```

```

        when wait_once_for_idle =>
            rdreq <= '0';
            state <= idle;

        end case;
    end if;
end process;

char_stack : entity work.stack
port map(
    clk => clk,
    char_in => stack_char,

    push => push,
    pop => pop,
    stack_full => stack_full,
    stack_empty => stack_empty,
    char_out => char_out
);

div_inst : entity work.division
port map(
    clock => clk,
    denom => "1010",
    numer => numer,
    quotient => quotient,
    remain => remain
);

end behavioral;

```

stack.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;

```

```

entity stack is
    port(
        clk :    in  std_logic;
        char_in : in  std_logic_vector(7 downto 0);

        push   : in  std_logic;
        pop    : in  std_logic;

        stack_full : out std_logic;
        stack_empty : out std_logic;
        char_out   : out std_logic_vector(7 downto 0)
    );
end stack;

architecture behavioral of stack is
    constant STACK_DEPTH : integer := 12; -- 10 digits + sign + margin
    type reg_array_type is array (0 to STACK_DEPTH-1) of std_logic_vector(7
downto 0);
    signal stack_mem      : reg_array_type := (others => (others => '0'));

    signal stack_head     : integer range 0 to STACK_DEPTH := 0;
begin

    -- Combinational outputs: no 1-cycle delay
    stack_empty <= '1' when stack_head = 0 else '0';
    stack_full  <= '1' when stack_head = STACK_DEPTH else '0';
    char_out    <= stack_mem(stack_head - 1) when stack_head > 0 else
(others => '0');

    process(clk) begin
        if rising_edge(clk) then
            if push = '1' then
                stack_mem(stack_head) <= char_in;
                stack_head <= stack_head + 1;
            elsif pop = '1' and stack_head > 0 then
                stack_head <= stack_head - 1;
            end if;
        end if;
    end process;
end process;

```

```
end behavioral;
```